



RUHR-UNIVERSITÄT BOCHUM

CACHES

Computer Architecture

Agenda

- What are Caches?
- Cache Addressing
- Replacement Policies
- Write Through versus Write Back/Write Allocation

What are Caches?

CPU vs. Memory



**Processor
Speed**

1 MHz

**Memory
Latency**

500 ns

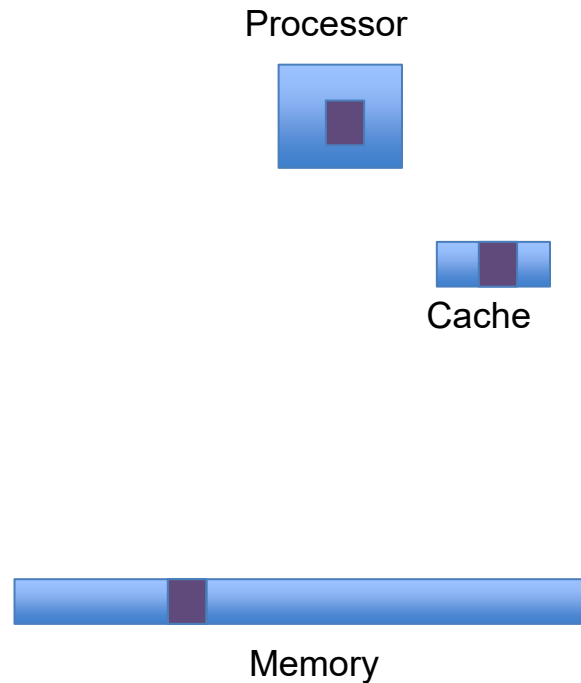


20×3700 MHz

50 ns

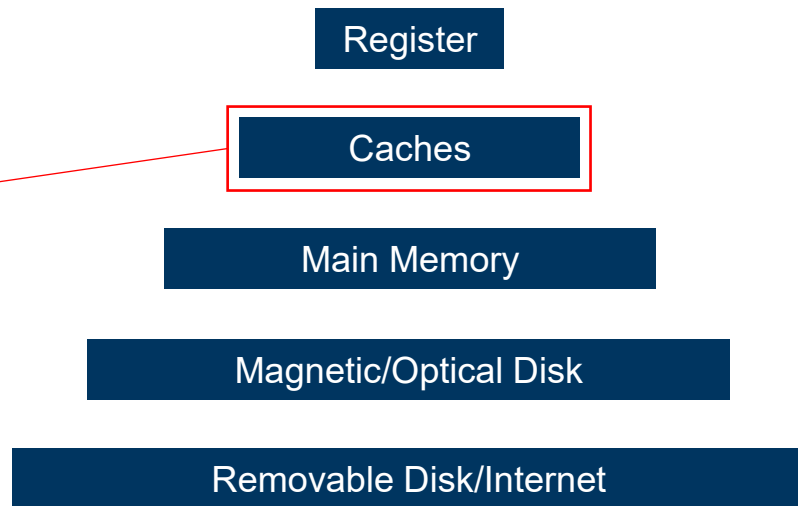
Caches

- Idea:
- The first time the processor needs data it brings it from memory
- It then stores the data nearby for future reuse
- The near storage is called a *cache*



Caches

User-transparent memory in which recently used data blocks are saved for reuse in the near future



Caches

SRAM cells - fast, but small

Operation: Memory and cache divided into fixed size *lines*. (16-128 bytes)

When accessing memory, check in cache.

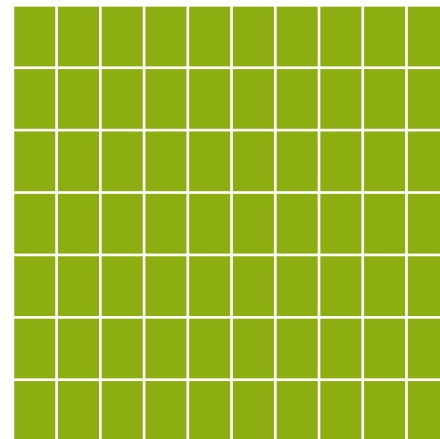
Cache hit: data found and is served from cache

Cache miss: data not in cache – load line from memory and keep a copy in cache

Cache



Main Memory



Principle of Locality

Why we can assume that data will be reused in the near future:

Temporal Locality: If an address is accessed once, it will very likely be accessed again soon

After an initial cache miss we expect multiple hits

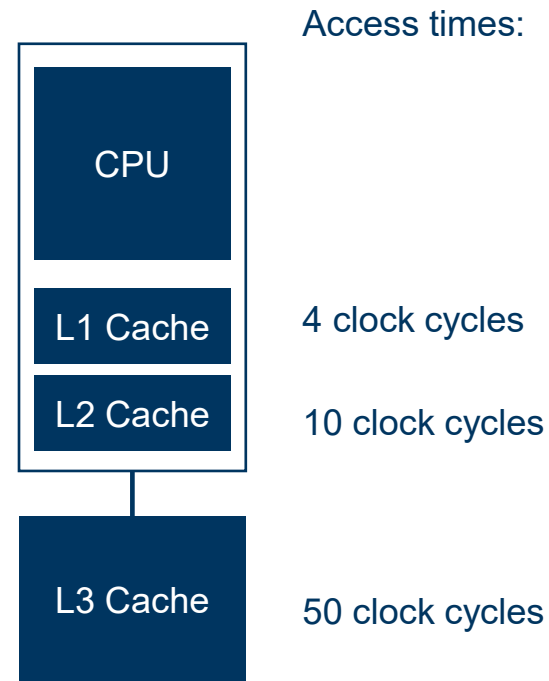
Spatial Locality: If an address is accessed, it is very likely that addresses close to it will be accessed too

Cache Hierarchy

Sometimes multiple caches are used:

The fastest cache is accessed first

If a cache miss occurs the next fastest cache is checked



Access Latency

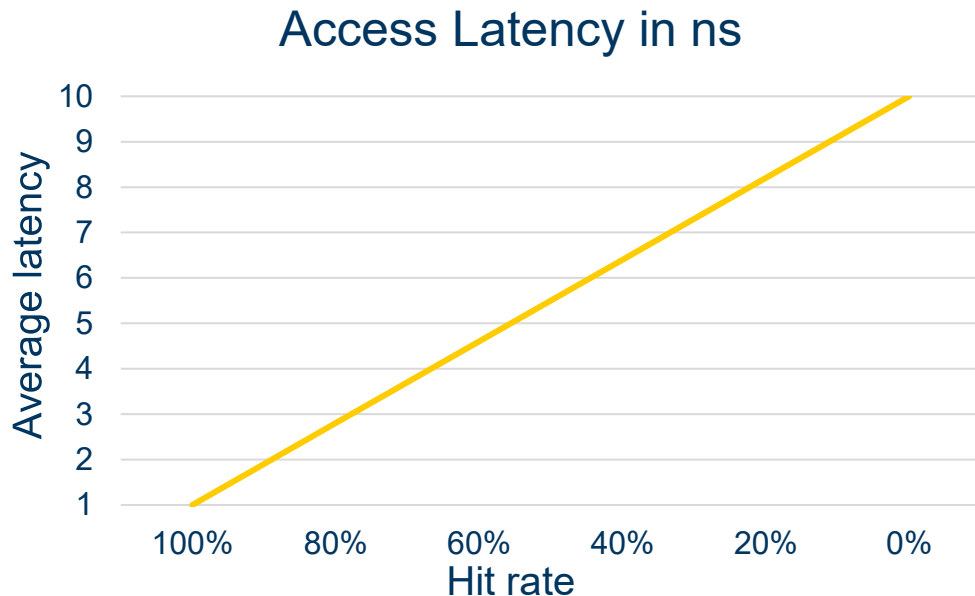
Cache access time: c

Cache hit rate: h

Main memory access time: m

Latency:

$$h \cdot c + (1 - h) \cdot (c + m) = \\ c + (1 - h)m$$



$$c = 1ns \quad m = 9ns$$

And now it's your turn...

To ensure that your latest program is cache-friendly, you ran extensive diagnostics and found out that out of 1256 data accesses, 867 were cache hits

The manufacturer of your main memory declares the memory access time as 80ns.

Assuming this data is a good representation of your machine, and the cache is ten times faster than your main memory, what is the average access latency?

Give your answer in nanoseconds.

Cache Latency – Why It Matters

```
#include <stdio.h>

char buffer[16384][16384] = {1};

int main() {
    int sum = 0;
    for (int i = 0; i < 16384; i++)
        for (int j = 0; j < 16384; j++)
            sum += buffer[i][j];
    printf("Sum is: %d\n", sum);
}
```

```
#include <stdio.h>

char buffer[16384][16384] = {1};

int main() {
    int sum = 0;
    for (int j = 0; j < 16384; j++)
        for (int i = 0; i < 16384; i++)
            sum += buffer[i][j];
    printf("Sum is: %d\n", sum);
}
```

Cache Addressing

Cache Addressing

- How can we find if data is in the cache or not?
- Search by address

Associative Caches

How can we find if data is in the cache?

Divide address to offset and tag

																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																					</
--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	----

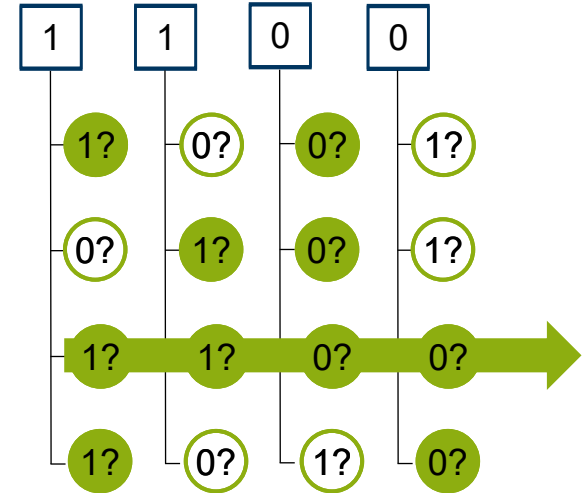
Cache line stores both tag and data

Search for tag to find data

Line	Tag	Data
0	0x12345678	0x7F55BA1
1	0xABCDEF1	0xFFBC6552
2	0x98765482	0x665006787
3	0x00000000	0xFFFFFFFF

Content Addressable Memory (CAM)

- A special type of memory.
- Each cell compares its contents to input.
- Output: row number where all cells match, if exists.
- Fast but expensive
 - Large area
 - High power consumption



Encode cache line number in address



Shorter tag – line number implicit

Simpler search – only need to match one tag

17 Direct-Mapped Caches | Caches

Direct-Mapped Cache: Example

- Assume offset is two bits
- Cache has four line
- Multiple locations map to same cache line
- Contention on cache space

Line number:

00 01 10 11

Cache Cell:



Memory:



Address:

1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
0000 0001 0100 0100 0100 0100 1000 1000 1000 1000 1100 1100 1100 1100 1100 1100

Set-Associative Caches

Cache consists of $S = 2^s$ sets

Set number encoded in address

Each set contains ω cache lines
(also called ways)

Uses associative search in sets

												12	11				6	5							0						
0	1	0	0	.	.	.	1	1	0	1	1	0	1	0	1	0	1	0	1	1	0	0	1	0	0	1					
												tag							set					offset							

Set 0	Set 1	Set 2	Set 3
Tag / Data	Tag / Data	Tag / Data	Tag / Data
Tag / Data	Tag / Data	Tag / Data	Tag / Data

Set-Associative Caches

A trade-off between associative and direct-mapped caches

An associative cache has one set

A direct-mapped cache has one way

Example: Intel Core L1 data cache

Cache line size: 64 bytes

Number of sets: 64

Number of ways: 8 or 12

Replacement Policies

Replacement Policies

When a cache set is full, how do we decide which line to replace?

Aim: reduce the number of cache misses

Idea: Replace the address that will not be needed for the longest time

Benefit: proven best policy – no policy achieves fewer misses

Downside: cannot be implemented in practice

Name: Clairvoyant Policy / Optimal Replacement Policy

Replacement Policies: FIFO

FIFO: First In First Out

Replace data that has been in the Cache the longest

Access history:

0x123, 0x236, 0x3A7, 0x49C, 0x581,
0x236, 0x678, 0x49C

0x581 replaces 0x123 because it
stayed the longest in the cache

Line 0	Line 1	Line 2	Line 3
<u>0x123</u>			
0x123	<u>0x236</u>		
0x123	0x236	<u>0x3A7</u>	
0x123	0x236	0x3A7	<u>0x49C</u>
<u>0x581</u>	0x236	0x3A7	0x49C
0x581	0x236	0x3A7	0x49C
0x581	<u>0x678</u>	0x3A7	0x49C
0x581	0x678	0x3A7	0x49C

Replacement Policies: LFU

LFU: Least Frequently Used

Replace data that has been used the fewest

If multiple entries have the same lowest usage frequency, there needs to be another replacement policy for tie-breaking

Access history:

0x123, 0x279, 0x123, 0x279, 0x315, 0x404, 0x500

0x500 replaces either 0x315 or 0x404. If we use FIFO as tie-breaking policy, it replaces 0x315

Line 0	Line 1	Line 2	Line 3
<u>0x123</u>			
0x123	<u>0x279</u>		
0x123	0x279		
0x123	0x279		
0x123	0x279	<u>0x315</u>	
0x123	0x279	0x315	<u>0x404</u>
0x123	0x279	<u>0x500</u>	0x404

Replacement Policies: LRU

LRU: Least Recently Used

Replace data that has been unused the longest

Access history:

0x145, 0x222, 0x30E, 0x4F0, 0x145, 0x503

0x503 replaces 0x222 because it hasn't been used for the longest time

Line 0	Line 1	Line 2	Line 3
<u>0x145</u>			
0x145	<u>0x222</u>		
0x145	0x222	<u>0x30E</u>	
0x145	0x222	0x30E	<u>0x4F0</u>
0x145	0x222	0x30E	0x4F0
0x145	<u>0x503</u>	0x30E	0x4F0

If We Use LRU:

We need note down when a cache entry was last used for every cache line

➡ This can take up a lot of space

We need to decide which entry hasn't been used for the longest

➡ This can take a lot of time

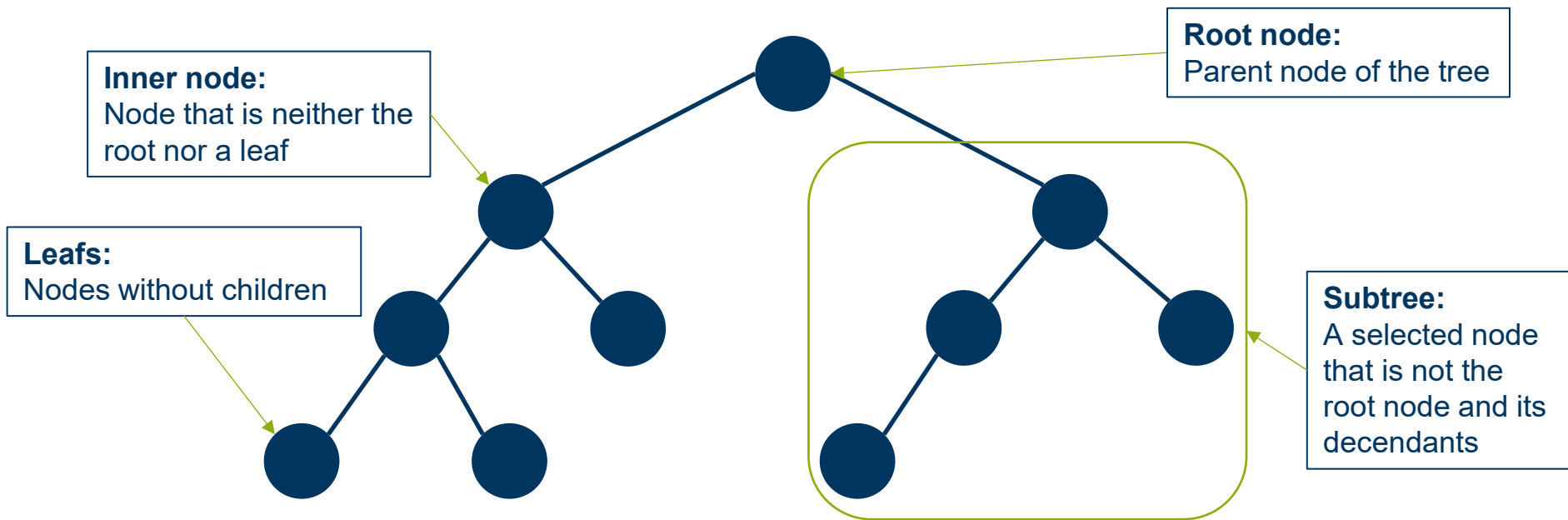
A more efficient Idea:

Implement a mechanism that keeps track of the oldest cache entry without a record on every entry

➡ Pseudo Least Recently Used (PLRU)

Excursion: Binary Trees

Important Criterion: Every node has at most two child nodes



Replacement Policies: Tree-PLRU

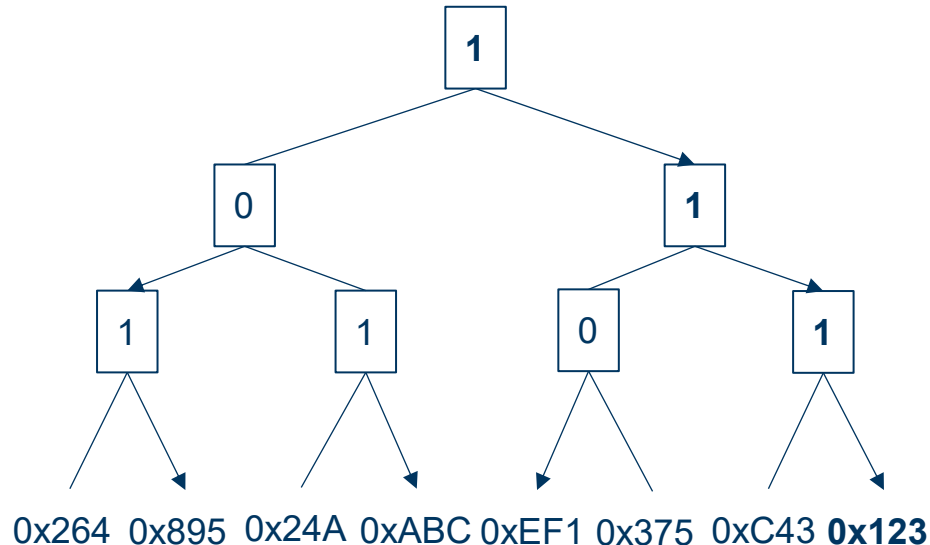
Cache structure is depicted as a binary tree

Leaves represent cache entries

Inner nodes indicate if the least recently used element is in the left or right child node:

0 - Left child

1 - Right child



Replacement Policies: Tree PLRU

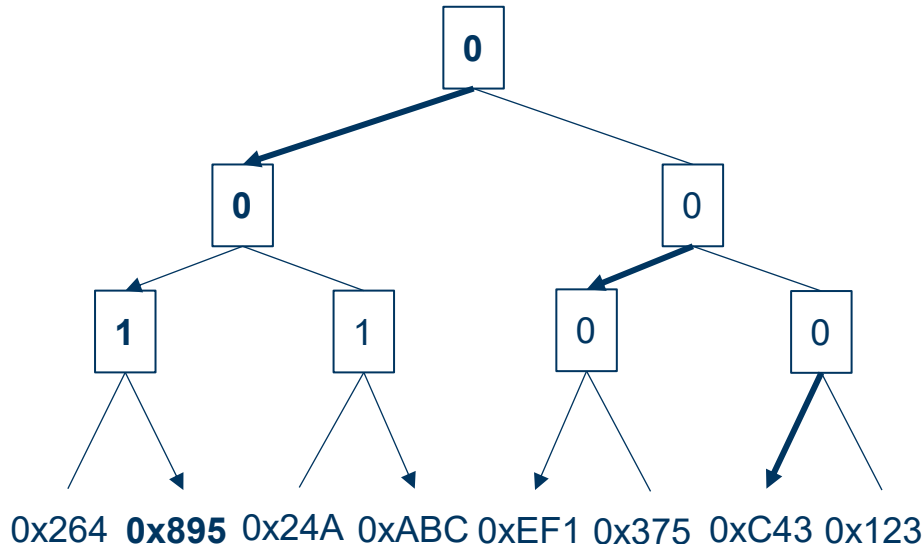
If an element is accessed the tree has to be updated

Every bit on the path from the root to the accessed leaf needs to be updated

Example:

Access 0x123

The new LRU-Element is 0x895



Replacement Policies: Bit-PLRU

Each Cache line has a status bit

Accessing a line sets the bit to 1

If the last status bit is set to 1 all other are set to 0

If a cache miss happens: leftmost line with status bit set to 0 is replaced

Replacement Policies: Bit-PLRU

Access history: 0x123, 0x234, 0x899, 0x100, 0x123, 0x200

Line 0	Status	Line 1	Status	Line 2	Status	Line 3	Status
0x123	1						
0x123	1	0x234	1				
0x123	1	0x234	1	0x899	1		
0x123	0	0x234	0	0x899	0	0x100	1
0x123	1	0x234	0	0x899	0	0x100	1
0x123	1	0x200	1	0x899	0	0x100	1

Bélády's Anomaly

Example:

Access history: 0x123, 0x456, 0x678, 0x890,
0x123, 0x456, 0x32A, 0x123, 0x456, 0x678,
0x890, 0x32A

Using a FIFO replacement policy, starting
from an empty cache, oldest element in bold.
Result: 9 cache misses (underlined)

**Would using more cache lines reduce the
number of misses?**

Line 0	Line 1	Line 2
<u>0x123</u>		
0x123	<u>0x456</u>	
0x123	0x456	<u>0x678</u>
<u>0x890</u>	0x456	0x678
0x890	<u>0x123</u>	0x678
0x890	0x123	<u>0x456</u>
<u>0x32A</u>	0x123	0x456
0x32A	0x123	0x456
0x32A	0x123	0x456
0x32A	<u>0x678</u>	0x456
0x32A	0x678	<u>0x890</u>
0x32A	0x678	0x890

And now it's your turn...

How many cache misses happen if we use four cache lines instead of three?

Access history: 0x123, 0x456, 0x678, 0x890, 0x123, 0x456, 0x32A, 0x123, 0x456, 0x678, 0x890, 0x32A

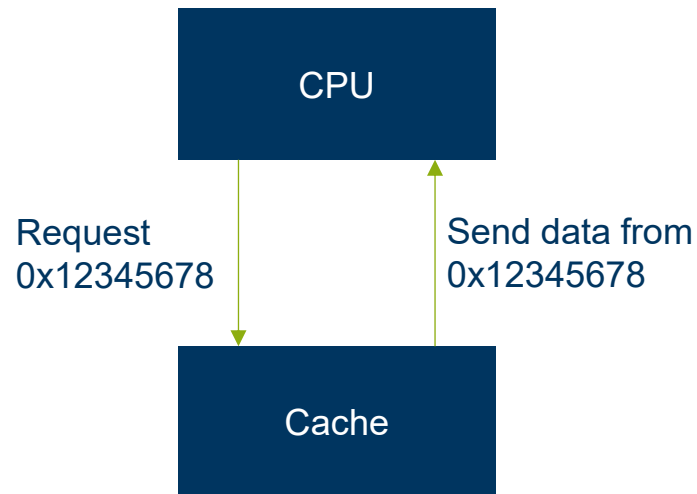
Cache Hits and Misses

Cache Hits

The data we look for is stored in the cache

➡ Read data directly from the cache

Faster than reading from memory

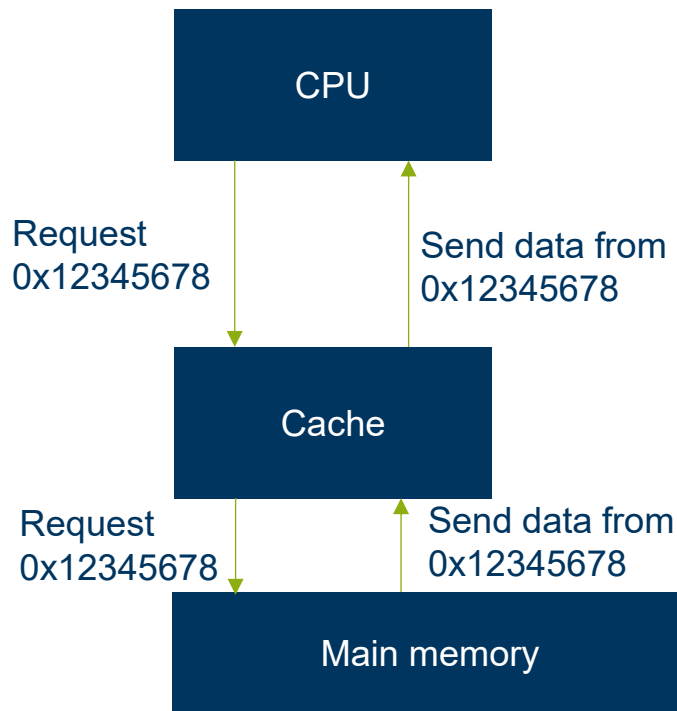


Cache Miss

The data we look for is not stored in the cache

➡ Load data into the cache from memory and then read it from the cache

Loading data into the cache takes time but we can assume that the extra time will amortize in the future



Types of Cache Misses: Compulsory

The data we look for has never previously been loaded into the cache

Example:

The Cache was empty before 0xAB4 was accessed

Line 0	Line 1	Line 2	Line 3
0xAB4			

Types of Cache Misses: Capacity

The data we look for has previously been loaded into the cache but had to be evicted because there was not enough space

Example:

We use a fully-associative cache and FIFO as replacement policy.

After the cache is filled completely, 0x73C replaces 0xCF6

When 0xCF6 is accessed again a capacity miss occurs

Line 0	Line 1	Line 2	Line 3
0xCF6			
0xCF6	0x56D		
0xCF6	0x56D	0xBF4	
0xCF6	0x56D	0xBF4	0x347
0x73C	0x56D	0xBF4	0x347
0x73C	0xCF6	0xBF4	0x347

Types of Misses: Conflict

The data we look for has previously been loaded into the cache, there is still free space in the cache but the data had to be evicted because there was not enough space in the available lines for the data

Example:

We use a set-associative cache with FIFO as replacement policy

After set 0 is filled completely, 0xB02 replaces 0xAB6

When we access 0xAB6 again, a Conflict Miss occurs, because there is still a free line in set 1

Set 0		Set 1	
Line 0	Line 1	Line 0	Line 1
0xAB6			
0xAB6	0x36E		
0xAB6	0x36E	0x29F	
0xB02	0x36E	0x29F	
0xB02	0xAB6	0x29F	

Write Through/Back

Keeping Data Consistent

The data in the cache is a copy of the data in main memory

Multiple copies can lead to inconsistencies

The cache and main memory need to be updated so that no faulty data accesses occur

We will now look at different techniques for doing that...

Write Policies

- How do we handle data stores when the data is cached?
- **Write through**
 - Write data to both cache and memory
- **Write back**
 - Initially, write data only to cache
 - Mark cache line as “dirty”
 - Eventually, update memory and remove dirty mark
 - Update must happen before line is evicted from the cache

Write Allocation

- How do we handle data stores when the data is **not** cached?
- **Write allocation**
 - Bring data to the cache
 - Follow with write back or write through
 - Usually used with write back
- **No write allocation**
 - Update memory, as in write through

Write Through versus Write Back/Write Allocation

Write trough:

- No issues with data inconsistencies
- If cache hit happens waiting for completion of write into main memory is necessary
- More traffic on system bus

Write back/write allocation:

- Data inconsistencies could happen if for example multiple processors use the same address space
- If cache hit (+cache miss if write allocation) happens writes can be done without waiting
- Less traffic on system bus

Summary

- Caches are user-transparent memory in which recently used and likely soon-to-be-used-again data is stored
- They are viable because the Principle of Locality applies to memory accesses in programs
- There are different types of Caches: Direct-Mapped, Associative and Set Associative
- If the used Cache is (partly) associative, a replacement policy is needed
- To keep the cache consistent with main memory while minimizing the impact on the system bus, different methods of updating data are used